Estructura de Dades: Pràctica 2

Pràctica 2. Estructura de dades Pila i Llista

1. Introducció

Objectius: Familiaritzar-se amb les estructures lineals
Temes de teoria relacionats amb la pràctica: Tema 3 Estructures de dades lineals bàsiques

2. Enunciat

En aquesta pràctica fareu la implementació i usareu dues estructures de dades lineals: una pila i una llista. Per la pila implementareu l'estructura de forma estàtica i per la llista fareu una implementació de l'estructura de forma dinàmica.

Exercici 1.

Definir i implementar el TAD (tipus abstracte de dades) **`PilaEstatica`** (pila en array) amb els següents membres i mètodes:

```
#ifndef PILA_ESTATICA_H
#define PILA_ESTATICA_H
#include <initializer_list>
#include <vector>
using namespace std;

template <class Elemento> class PilaEstatica {
public:
  PilaEstatica(int max = TAMANO_MAXIMO);      // Constructor
  PilaEstatica(const PilaEstatica &original); // Constructor copia
  PilaEstatica( initializer_list<Elemento> elementos); //Inicializa la pila con una lista de elementos
  PilaEstatica(const vector<Elemento>& elementos); // Inicializa la pila a partir de un vector
  int tamano() const;      // Devuelve la longitud de la pila
  bool estaVacia() const;  // Devuelve true si la pila está vacía
  bool estaLlena() const;  // Devuelve true si la pila está llena
  const Elemento &elementoEncima() const; // Devuelve el elemento de encima, exception si está vacía
  void anadirElemento(const Elemento &e); // Añade un nuevo elemento encima de
                                          // la pila, exception si no es posible
  void suprimirElemento(); // Elimina el elemento de encima de la pila,
                           // exception si no es posible
  void print() const; // Imprime la pila completa por stdout
private:
  enum { TAMANO_MAXIMO = 30 };
  vector<Elemento> _datos;
  int _capacidad;
  int _encima; // Posición del elemento de encima (el top) de la pila
};

#endif // PILA_ESTATICA_H
```

Estructura de Dades: Pràctica 2

A sobre teniu la definició de l'especificació de la classe PilaEstatica.

- **NO** podeu modificar el .h de la classe PilaEstatica, ni afegir més mètodes o atributs.
- La implementació es farà a sota de la definició de la classe, en el PilaEstatica.h

Realitzeu la implementació del TAD PilaEstatica. Tot seguit, codifiqueu un *main.cpp* que cridi a una funció, `casProvaEx1()`, que serà el cas de prova 1.

Cas de prova 1:

Pas	Entrada	Sortida
1	Crear una PilaEstatica de mida 3	Estructura creada
2	Inserir element 10	Element 10 agregat
3	Inserir element 11	Element 11 agregat
4	Imprimir front	Front: 11
5	Inserir element 12	Element 12 agregat
6	Inserir element 13	EXCEPTION: L'estructura està plena
7	Imprimir PilaEstatica	[10, 11, 12]
8	Imprimir front	Front: 12
9	Suprimir element	Element 12 eliminat
10	Inserir element 14	Element 14 agregat
11	Imprimir PilaEstatica	[10, 11, 14]
12	Imprimir front	Front: 14
13	Suprimir element	Element 14 eliminat
14	Suprimir element	Element 11 eliminat
15	Suprimir element	Element 10 eliminat
16	Suprimir element	EXCEPTION: L'estructura està buida

Recordeu que és necessari recollir les possibles excepcions al main i fer-ne la gestió de les mateixes indicant el missatge corresponent per pantalla.

Afegiu el cas de prova com un mètode `casProvaEx1()` en el main. A continuació, afegiu una altra funció al main, `casProvaMenu()`, que tindrà un *menú* amb les següents opcions:

1. Inserir element a la pila
2. Suprimir element de la pila
3. Consultar el top de la pila
4. Imprimir tot el contingut de la PilaEstatica
5. Imprimir la posició del top de la pila
6. Sortir

El main us quedarà:

```
int main() {
  casProvaEx1();
  casProvaMenu();

  return 0;
```

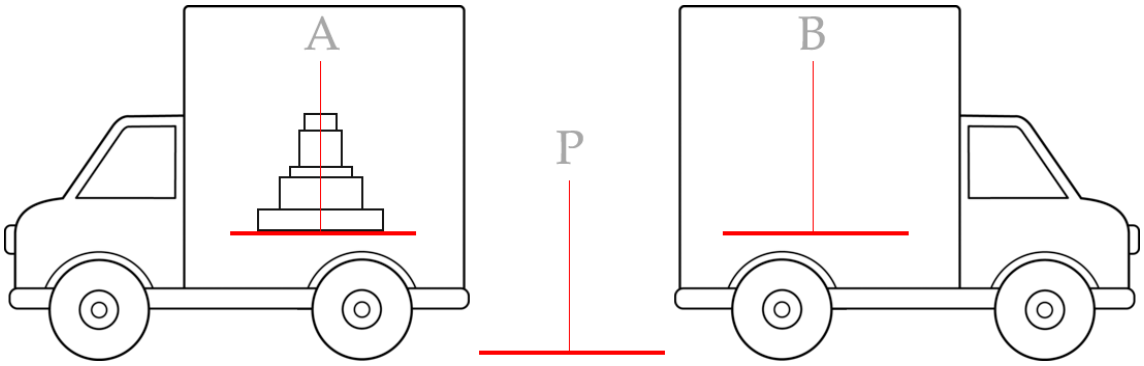
```
}
```

Important:

- En aquest primer exercici s'ha de fer el TAD `PilaEstatica` amb templates
- Assumim que els elements de la `PilaEstatica` poden ser de tipus enter, string o double
- El constructor crea un vector de mida TAMANO_MAXIMO o la capacitat indicada al constructor
- Les excepcions que retorneu seran de la llibreria stdexcept, com per exemple out_of_range o runtime_error

Exercici 2.

Afegiu en el **main.cpp** de l'exercici 1, una nova funció, `casProvaEx2()`, que resolgui, mitjançant 3 piles de tipus `PilaEstatica`, el problema que se li planteja a una empresa de missatgeria.



L'empresa vol implementar un braç robòtic que mogui que tots els paquets transportats del camió A al camió B. Tal i com es veu a la figura (número de paquets N=5). El braç robòtic pot moure els paquets directament del **camió A** al **camió B** o del **camió A** pot deixar-los a la **passarel·la P** que hi ha situada entre ambdós camions. Per facilitar les coses, cada paquet tindrà assignat un identificador enter entre 1 i N en funció de la mida de la seva base: el paquet amb la base més petita tindrà l'identificador 1 i el de la base més gran N. A l'hora de realitzar la tasca, el braç robòtic ha de tenir en compte les següents restriccions:

- Els paquets només es poden moure d'un en un, agafant sempre el de la part superior d'una de les piles de paquets. Restricció, cada camió només té una pila de paquets.
- Per qüestions de seguretat durant el transport, els paquets amb el costat més llarg (ID més gran) no poden quedar apilats a sobre d'altres que tinguin el costat més curt (ID més petit).

Noteu que definirem 3 piles d´IDs de paquets: A, B i P. Llavors, l'acció "moure paquet" consisteix en treure l'ID del paquet superior d'una de les piles i col·locar-lo al damunt d´una de les piles restants sobre els IDs que ja hi puguin haver en la pila de destí (sempre complint les restriccions especificades anteriorment).

El programa implementat ha de:

- Permetre a l'usuari especificar el nombre de paquets:

```
Hola, sóc el braç robòtic.
Introdueix el nombre de paquets: 3
```

- Inicialitzar les piles. Per la inicialització de la pila A, cal que feu servir el tercer constructor del TAD. Tingueu en compte que, un cop inicialitzada, l´ID més petit sigui el primer en ser desapilat, el segon ID més petit sigui el segon en ser desapilat, etc.

- Mostrar per pantalla l'estat inicial de les piles de paquets utilitzant el mètode print() del TAD:

```
L'estat inicial és el següent:
A: 3, 2, 1
P:
B:
```

- El següent pseudo-codi indica com es podem moure cada paquet (n) del camió A al camió B, passant per la passarel·la P.

```
Funció logistica(n, camioA, camioB, passarel·la)
  si n=1 , llavors moure un paquet del camioA  al camioB
  sinó
    cridar logistica(n-1, camioA, passarel·la, camioB)
    cridar logistica(1, camioA, camioB, passarel·la)
    cridar logistica(n-1, passarel·la, camioB, camioA)
  fiSi
fiFunció
```

- Mostrar per pantalla l'estat quan es mouen els paquets de les piles:

```
L'estat intermig és el següent:

A: Pila = [3, 2]  mida= 2

P: Pila = []  mida = 0

B: Pila = [1]  mida = 1

L'estat intermig és el següent:

A: Pila = [3]  mida = 1

P: Pila = [1]  mida = 1

B: Pila = [2]  mida = 1

L'estat intermig és el següent:

A: Pila = []  mida = 0

P: Pila = [3]  mida = 1

B: Pila = [2, 1]  mida = 2

L'estat intermig és el següent:

A: Pila = []  mida = 0

P: Pila = [2, 1]  mida = 2

B: Pila = [3]  mida = 1

L'estat intermig és el següent:

A: Pila = [2]  mida = 1
```

```
P: Pila = [3]  mida = 1

B: Pila = [1]  mida = 1

L'estat intermig és el següent:

A: Pila = []  mida = 0

P: Pila = [1]  mida = 1

B: Pila = [3, 2]  mida = 2

L'estat intermig és el següent:

A: Pila = []  mida = 0

P: Pila = []  mida = 0

B: Pila = [3, 2, 1]  mida = 3
```

- Mostrar per pantalla l'estat final de les piles:

```
L'estat final és el següent:
A:
P:
B: 3, 2, 1
```

El main.cpp us quedarà:

```
int main() {
  casProvaEx1();
  casProvaMenu ();
  casProvaEx2();
  return 0;
}
```

Exercici 3

Implementar el TAD **LinkedList**, que implementa una llista per posició amb encadenaments dobles. Es proposa fer-ho mitjançant l'ús de *templates*.

Aquesta LinkedList està formada per nodes de tipus TAD NodeList. Aquest TAD NodeList conté com a mínim tres atributs: `_element` on es guarda l'element a inserir a la LinkedList, `_previous` que és l'apuntador al node anterior i `_next` que és l'apuntador al següent node de la LinkedList.

L'especificació amb el mínim d'operacions necessàries del TAD NodeList és la següent:

- **`constructor:`** fareu tres constructors, el constructor per defecte, un per paràmetres que construeix el node amb l'element que rep com a paràmetre, i un de còpia
- **`accessElement`**: retorna l'element que hi ha guardat al node
- **`accessNext`**: retorna l'adreça del següent node o *`nullptr`* en cas que no hi hagi següent
- **`setNext`**: modifica l'adreça de `_next` per l'adreça rebuda com a paràmetre
- **`accessPrevious`**: retorna l'adreça del node anterior o *`nullptr`* en cas que no hi hagi anterior
- **`setPrevious`**: modifica l'adreça de `_previous` per l'adreça rebuda com a paràmetre

D'altra banda, el TAD LinkedList es defineix tal i com es proposa a continuació:

```
template <class Element>
class LinkedList
{
public:
   LinkedList();
   LinkedList(initializer_list<Element> elements);
   LinkedList(const LinkedList& origen);
   virtual ~LinkedList();
   int size() const;
   bool isEmpty() const;
   Position<Element> beginning() const; // return first position
   Position<Element> end() const; // return last position
   void insertAfter(Position<Element> & position, const Element& element);
   void insertBefore(Position<Element>& position, const Element& element);
   void insertBeginning(const Element& element);
   void insertEnd(const Element& element);
   void deletePosition(Position<Element>& position);
   void print() const;
private:
   NodeList<Element>* _head;
   NodeList<Element>* _tail;
   int _size;
};
```

Fixeu-vos en alguns detalls d'aquest TAD LinkedList:

- La LinkedList s'ha definit amb nodes de tipus *NodeList*, que són bidireccionals.
- En aquesta pràctica **es demana la implementació de la llista per posició amb nodes *NodeList* bidireccionals i que es faci amb dos sentinelles**. Un al davant i un altre al darrere de tot. Els elements de la *LinkedList* se situaran entre els dos sentinelles. Això facilitarà la implementació de les diferents funcions de la LinkedList.
- La funció ***size*** retorna el nombre d'elements de la LinkedList en aquest moment.
- La funció ***isEmpty*** retorna *true* si la LinkedList està buida, *false* en cas contrari.
- Noteu que la LinkedList, amb els *NodeLlista* bidireccionals, permet fer un recorregut de la LinkedList cap endavant i cap endarrere. En aquest cas, només farem recorreguts endavant. Les funcions que ens permeten fer el recorregut són les següents:
    - **beginning**: retorna el següent element al sentinella de l'inici de la LinkedList.
    - **end**: retorna el darrer sentinella de la LinkedList
- Es pot inserir tant al principi de la LinkedList amb ***insertBeginning*** com al final d'aquesta amb ***insertEnd***. La inserció sempre es realitza entre els dos sentinelles. És a dir, *insertBeginning* inserta davant de tots els elements i darrera del primer sentinella i *insertEnd* inserta al final de tots elements i davant del darrer sentinella.
- Cal implementar el ***constructor de còpia***. Aquest ha de duplicar la llista encadenada, de forma que els espais de memòria dels nodes de la llista des d'on es copia i de la llista final siguin diferents. No n'hi ha prou amb copiar la direcció dels punters *_head* i *_tail*, sinó que cal fer-ne un de completament nou per cadascun dels elements de la LinkedList original.

A més a més, el TAD *LinkedList* utilitza el TAD Position. Aquest TAD es defineix tal i com es proposa a continuació:

```
template <class Element>
class Position
{
public:
   Position(NodeList<Element>* node); //inicialitza la classe Position amb el node que rep com a paràmetre.
   Position(const Position& origen);
   virtual ~Position();
   Position<Element> next() const; //retorna la següent posició a l'actual.
                                   // No es modifica el contingut de l'actual posició.
                                   // Si no hi ha següent posició, aquest mètode ha de llençar una excepció.
   Position<Element> previous() const; // retorna l'anterior posició
   const Element& element() const; // retorna el contingut de l'actual posició
   NodeList<Element>* deletePosition();
   void setPrevious(NodeList<Element>* node); // assigna l'adreça de l'anterior node
   void setNext(NodeList<Element>* node); // assigna l'adreça del següent node
   Position<Element> operator++() const; // sobrecarrega operador ++
   Position<Element> operator--() const; // sobrecarrega operador --
   bool operator==(const Position<Element> & other) const; // sobrecarrega operador ==
   bool operator!=(const Position<Element> & other) const; // sobrecarrega operador !=
   const Element & operator*() const; // sobrecarrega operador contingut
private:
   NodeList<Element> * _node;
};
```

A continuació es detalla el comportament d'aquestes funcions.

- **Constructor**: inicialitza la classe *Position* amb el node que rep com a paràmetre.
- **next**: retorna la següent posició a l'actual. No es modifica el contingut de l'actual posició. Si no hi ha següent posició, aquest mètode ha de llençar una excepció.
- **previous**: retorna l'anterior posició a l'actual. No es modifica el contingut de l'actual posició. Si no hi ha anterior posició, aquest mètode ha de llençar una excepció.
- **element**: retorna l'element que hi ha guardat en la posició actual.
- **operator++**: retorna la següent posició a l'actual.
- **operator==**: retorna cert si l'adreça de *other* és la mateixa que la posició actual.
- **operator!=**: retorna cert si l'adreça de *other* és diferent a la posició actual.
- **operator***: retorna l'element que hi ha guardat en la posició actual.

Per acabar, en el mètode **main.cpp** fer dues crides a dos casos de prova de la LinkedList de forma que es provin tots els mètodes definits a l'especificació del TAD.

```
int main(int argc, char** argv)
{
   casProva1(); // prova amb dues llistes d'enters
   casProva2(); // prova amb una llista d'strings

   return 0;
}
```

```
void casProva1()
{  LinkedList<int> list1;
   list1.insertBeginning(5);
   list1.insertBeginning(3);
   list1.insertEnd(7);
   list1.insertEnd(8);
   list1.print();
try{
   Position<int> pos = list1.beginning();
   pos = list1.beginning();  list1.insertAfter(pos, 4);  list1.print();
   pos = list1.beginning();  list1.insertBefore(pos, 2); list1.print();
   pos = list1.end();        list1.insertBefore(pos, 9); list1.print();
   pos = list1.beginning();  list1.insertAfter(pos, 5);  list1.print();
   pos = list1.end();        list1.insertAfter(pos, 8);  list1.print();

}catch (std::exception& e)  {
    std::cout << "Error: " << e.what() << std::endl;
 }
 int posicio, element;
 cout << " A quina posicio vols inserir" << endl;
 cin >> posicio;

 cout << " Quin element vols inserir" << endl;
 cin >> element;
    try {
            auto itr = list1.beginning();
            for (int i = 0; i < posicio; ++i)
            {
                itr = ++itr;
            }
            list1.insertBefore(itr, element);
            list1.insertAfter(itr, element + 10);
        }
        catch (std::exception& e)  {
            std::cout << "Error: " << e.what() << std::endl;
        }
   list1.print();
   cout << " Segona part de la prova 1 de la llista " << endl;
   LinkedList<int> list2 = {1, 2, 3, 4, 5};
   list2.print();
   Position<int> pos2 = list2.beginning();
   list2.deletePosition(pos2);
   list2.print();
   pos2 = list2.beginning();
   pos2 = ++pos2;
   list2.deletePosition(pos2);
   list2.print();
}
```

Un cop heu completat i us funciona el cas prova 1, cal fer el `casProva2()`, en aquesta funció definiu una `LinkedList` d'strings, ompliu amb diferents elements aquesta llista i utilitzeu tots els mètodes definits a la classe LinkedList, incloent-hi el mètode imprimir. Finalment, feu un recorregut de la llista per posició, des de l'inici de la llista fins al final de la llista en ordre d'esquerra a dreta de la llista. A continuació teniu un exemple d'execució del main.cpp.

```
Introdueix quantes paraules vols inserir: 5
Introdueix una paraula: gos
Inserir al principi? (s/n) s
Introdueix una paraula: gat
Inserir al principi? (s/n) n
Inserir al final? (s/n) s
Introdueix una paraula: lloro
Inserir al principi? (s/n) n
Inserir al final? (s/n) n
A quina posició vols inserir? 1
Inserir abans de la posició? (s/n) s
Introdueix una paraula: serp
Inserir al principi? (s/n) n
Inserir al final? (s/n) s
Introdueix una paraula: hàmster
Inserir al principi? (s/n) n
Inserir al final? (s/n) n
A quina posició vols inserir? 2
Inserir abans de la posició? (s/n) n
Iteració dels elements endavant:
gos
lloro
gat
hàmster
serp
```

Preguntes a contestar a l'inici del main (un comentari a dalt de tot del main.cpp):

1. Tenint en compte la teva implementació del TAD `LinkedList`, indica per a cadascuna de les operacions del TAD Llista quin és el seu cost computacional teòric. Justifica la resposta.
2. Creieu que la classe NodeList hauria estat millor implementar-la amb encadenaments simples? Justifica la teva resposta.

Important:

- En aquest tercer exercici el TAD `LinkedList` serà amb templates
- No podeu modificar i/o ampliar l'especificació de la classe `LinkedList`

3. Lliurament

Implementar els exercicis en C++ i usar el **Visual Studio Code amb C++ versió 11**. Creareu un projecte de Visual Studio Code per a l'exercici 1 i un altre per a l'exercici 3. Lliurar el codi C++ corresponent als vostres exercicis en una **carpeta anomenada codi**, amb una **subcarpeta per a cada exercici**. Els comentaris de cada exercici (observacions, decisions preses i resposta a les preguntes plantejades, si n'hi ha) els fareu com a comentari al fitxer main.cpp de cada exercici.

Com a màxim el dia del lliurament es penjarà en el campus virtual un fitxer comprimit **en format ZIP** amb el nom del grup (A, B, C, D, E o F), el nom i cognoms de la persona que ha fet la pràctica i el número de la pràctica com a nom de fitxer. Per exemple, **GrupA_BartSimpson_P2.zip**, on P2 indica que és la "pràctica 2". El fitxer ZIP inclourà: la carpeta amb tot el codi.

Els criteris per acceptar la pràctica són: La pràctica ha de compilar i executar en la seva totalitat. I, la pràctica ha de ser orientada a objectes. Si una part no us funciona, comenteu el codi.

IMPORTANT: La còpia de la implementació de la pràctica implica un zero a la nota de pràctiques de l'assignatura per les persones implicades (tant la que ha copiat com la que ha deixat copiar).

4. Planificació

Per aquesta pràctica disposeu de tres setmanes. Els professors us proposem la següent planificació:

- **Setmana 1 (del 2 de març al 8 de març) - LAB 4** amb el professor
  - **EXAMEN PRÀCTICA 1**
  - Implementació de l'exercici 1
- **Setmana 2 (del 9 de març al 15 de març) - LAB 5** amb el professor
  - Implementació de l'exercici 2
  - Implementació de l'exercici 3
- **Setmana 3 (del 16 de març al 20 de març) - LAB 6** amb el professor
  - Implementació de l'exercici 3 i comentar el codi
  - Tasca al campus virtual per lliurar tota la pràctica

El lliurament final de la pràctica serà el **dia 22 de març de 2026** al campus virtual. Recordeu que s'han de lliurar tots els exercicis en el **fitxer .zip** al campus virtual.

La Setmana del 23 de març al 27 de Març teniu la prova pràctica de la pràctica 2 al LAB7.

- És obligatori lliurar la pràctica al campus virtual la setmana anterior
- La prova consistirà en una modificació o ampliació de la pràctica lliurada